

C++ 기초 알고리즘

이진 탐색 완전 입문

binary_search · lower_bound · upper_bound

정렬된 데이터에서 값을 빠르게 찾는 원리부터 직접 구현, STL 사용법, 중복 원소 처리까지

대상: C++ 문법을 조금 배웠지만 알고리즘은 처음인 학습자

1. 먼저 알아야 할 것

탐색(search)은 여러 데이터 중 원하는 값을 찾는 작업입니다. 예를 들어 배열 {1, 3, 5, 7, 9}에서 7의 위치를 찾는 것입니다.

선형 탐색: 처음부터 하나씩 확인

```
vector<int> a = {1, 3, 5, 7, 9};
for (int i = 0; i < a.size(); i++) {
    if (a[i] == 7) cout << i;    //    : 3
}
```

원소가 N개면 최악의 경우 N개를 모두 확인합니다. 시간 복잡도는 $O(N)$ 입니다.

이진 탐색: 가운데를 보고 절반을 버리기

정렬된 배열에서는 가운데 값과 target을 비교해 한쪽 절반을 통째로 버릴 수 있습니다.

단계	남은 범위	가운데 판단
1	1 3 5 7 9 11 13	$7 < 11 \rightarrow$ 왼쪽 절반 제거
2	9 11 13	$11 == 11 \rightarrow$ 발견

핵심 전제: 데이터가 반드시 정렬되어 있어야 합니다. 범위가 매번 절반이 되므로 시간 복잡도는 $O(\log N)$ 입니다.

인덱스란?

C++ 배열과 vector의 위치 번호는 0부터 시작합니다. 원소가 5개면 인덱스는 0, 1, 2, 3, 4이고 마지막 인덱스는 `size() - 1`입니다.

2. 이진 탐색 직접 구현

```
int binarySearch(const vector<int>& a, int target) {
    int left = 0;
    int right = static_cast<int>(a.size()) - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (a[mid] == target) return mid;
        if (a[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

변수 세 개

변수	뜻
left	현재 탐색 범위의 왼쪽 끝
right	현재 탐색 범위의 오른쪽 끝
mid	현재 범위의 가운데

왜 이렇게 움직일까?

- $a[mid] == target$: 값을 찾았으므로 mid를 반환합니다.
- $a[mid] < target$: target은 오른쪽에 있으므로 $left = mid + 1$ 입니다.
- $a[mid] > target$: target은 왼쪽에 있으므로 $right = mid - 1$ 입니다.
- $left > right$ 가 되면 남은 범위가 없으므로 -1을 반환합니다.

mid는 $left + (right - left) / 2$ 로 계산하는 습관을 들이세요. $(left + right) / 2$ 보다 정수 오버플로에 안전합니다.

3. 중복 값과 경계 탐색

배열 {1, 2, 2, 2, 4, 7}에서 일반 이진 탐색으로 2를 찾으면 어느 2가 선택될지 보장되지 않습니다. 중복 구간의 시작과 끝이 필요할 때 lower_bound와 upper_bound를 사용합니다.

lower_bound

target 이상인 첫 번째 위치를 찾습니다. target=2이면 인덱스 1, target=3이면 값 4가 있는 인덱스 4입니다.

```
int lowerBound(const vector<int>& a, int target) {
    int left = 0, right = static_cast<int>(a.size());
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (a[mid] < target) left = mid + 1;
        else right = mid;
    }
    return left;
}
```

upper_bound

target보다 큰 첫 번째 위치를 찾습니다. target=2이면 모든 2 다음의 값 4, 즉 인덱스 4입니다.

```
int upperBound(const vector<int>& a, int target) {
    int left = 0, right = static_cast<int>(a.size());
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (a[mid] <= target) left = mid + 1;
        else right = mid;
    }
    return left;
}
```

4. lower와 upper의 차이

함수	찾는 위치	target=2 결과
lower_bound	2 이상인 첫 위치	1
upper_bound	2보다 큰 첫 위치	4

```
:      1  2  2  2  4  7
:      0  1  2  3  4  5
           ^      ^
        lower  upper
```

같은 값의 개수

중복된 target들은 [lower, upper) 구간에 있습니다. 대괄호 쪽은 포함하고 소괄호 쪽은 포함하지 않는다는 뜻입니다.

```
int count = upperBound(a, target) - lowerBound(a, target);
// target=2: 4 - 1 = 3
```

직접 구현에서 right가 size()인 이유

경계 탐색은 [left, right) 형태의 반열린 구간을 사용합니다. right 자체는 포함하지 않습니다. 조건을 만족하는 값이 없으면 size()가 자연스럽게 반환되어 배열 끝을 나타냅니다.

lower_bound 조건은 $a[mid] < target$, upper_bound 조건은 $a[mid] \leq target$ 입니다. upper_bound는 같은 값까지 모두 지나가야 하므로 등호가 추가됩니다.

5. STL 표준 함수 사용법

실전에서는 직접 구현보다 <algorithm>의 검증된 함수를 주로 사용합니다.

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> a = {1, 2, 2, 2, 4, 7};
    int target = 2;

    bool exists = binary_search(a.begin(), a.end(), target);
    int lower = lower_bound(a.begin(), a.end(), target) - a.begin();
    int upper = upper_bound(a.begin(), a.end(), target) - a.begin();

    cout << boolalpha << exists << '\n'; // true
    cout << lower << '\n';                // 1
    cout << upper << '\n';                // 4
    cout << upper - lower << '\n';        // 3
}
```

begin(), end(), 반복자

- a.begin(): 첫 번째 원소의 위치
- a.end(): 마지막 원소 바로 다음 위치
- lower_bound의 반환값은 인덱스가 아니라 반복자(iterator)
- 반환 반복자에서 a.begin()을 빼면 인덱스가 됩니다.

값 자체를 읽을 때

```
auto it = lower_bound(a.begin(), a.end(), target);
if (it != a.end()) {
    cout << *it; // it
} else {
    cout << " ";
}
```

6. 입력, 정렬, 탐색 전체 예제

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int& x : a) cin >> x;

    sort(a.begin(), a.end());

    int target;
    cin >> target;

    auto lower = lower_bound(a.begin(), a.end(), target);
    auto upper = upper_bound(a.begin(), a.end(), target);

    if (lower == upper) {
        cout << "          .\n";
    } else {
        cout << "          : " << lower - a.begin() << '\n';
        cout << "          : " << upper - a.begin() - 1 << '\n';
        cout << "          : " << upper - lower << '\n';
    }
}
```

예시 입력과 결과

입력	정렬 후	출력
6 / 7 2 1 2 4 2 / 2	1 2 2 2 4 7	첫 1, 마지막 3, 개수 3

sort의 시간 복잡도는 $O(N \log N)$ 입니다. 한 번 정렬한 뒤 여러 번 탐색한다면 각 탐색은 $O(\log N)$ 이므로 효율적입니다.

7. 자주 하는 실수

실수	문제	해결
정렬하지 않음	이진 탐색 결과를 신뢰할 수 없음	먼저 sort 실행
left = mid	범위가 줄지 않아 무한 반복 가능	버릴 때 mid + 1
end()를 역참조	존재하지 않는 위치 접근	it != a.end() 검사
upper를 마지막 위치로 착각	upper는 마지막 다음 위치	마지막 = upper - 1
빈 vector 무시	size()-1 변환 주의	직접 구현 시 int 변환 후 -1

선택 기준 한 장 요약

하고 싶은 일	사용할 함수
값이 있는지만 확인	binary_search
target이 들어갈 첫 위치	lower_bound
target보다 큰 첫 위치	upper_bound
target의 개수	upper_bound - lower_bound
첫 target 인덱스	lower_bound 후 존재 검사
마지막 target 인덱스	upper_bound - 1 후 존재 검사

최종 암기 문장

binary_search는 존재 여부, lower_bound는 target 이상인 첫 위치, upper_bound는 target 초과인 첫 위치.

8. 연습문제와 정답

문제

- ① {1, 3, 3, 3, 8}에서 target=3의 lower와 upper는?
- ② 위 배열에서 3의 개수는?
- ③ {2, 5, 9}에서 lower_bound(6)은 어느 인덱스인가?
- ④ {2, 5, 9}에서 upper_bound(9)는 어느 인덱스인가?
- ⑤ 이진 탐색 전에 반드시 만족해야 하는 조건은?

정답

- ① lower=1, upper=4
- ② 4-1=3개
- ③ 인덱스 2 (값 9)
- ④ 인덱스 3, 즉 end()
- ⑤ 탐색 범위가 같은 기준으로 정렬되어 있어야 합니다.

마지막 체크리스트

- ☐ 데이터가 정렬되었는가?
- ☐ 존재 여부와 위치 중 무엇이 필요한가?
- ☐ 반환값이 end()일 수 있는가?
- ☐ upper_bound가 가리키는 곳은 마지막 원소의 다음 위치임을 기억했는가?
- ☐ 중복 개수는 upper - lower로 구했는가?

처음에는 코드를 외우기보다 '가운데를 확인하고 필요 없는 절반을 버린다'는 그림을 떠올리세요. 경계 탐색에서는 '조건을 만족하는 첫 위치'를 끝까지 좁힌다고 생각하면 됩니다.